🍕 FoodReady - Modern Multi-Role Food Delivery Web Application

An end-to-end, multi-vendor food ordering and dispatch platform built on **ASP.NET MVC 5 (.NET Framework 4.8.1)**, **SQL Server**, and **ADO.NET Stored Procedures**. It supports a complete 4-role ecosystem (Customer, Restaurant Owner, Kitchen Staff, and Delivery Partner) backed by a comprehensive Executive Administration control panel.

📌 Table of Contents

- Core Features & Overview
- Technology Stack
- Architecture & Project Structure
- User Roles & Complete Journeys
- Customer Experience
- Restaurant Owner Portal
- Kitchen Staff / Chef Portal
- Delivery Partner / Rider Portal
- System Administrator Control Center
- Intelligent Backend Algorithms & Engines
- Two-Stage Smart Rider Allocation
- Live 8-Stage Order Tracking & Route Map
- 6-Digit Secure Delivery OTP Handover
- 5-Minute Inactive Order Auto-Rejection
- Multi-Vendor Cart Splitting
- Database Schema & Scripts
- Default Seed Credentials for Testing
- How to Run & Setup Locally

🌟 Core Features & Overview

- **Unified Multi-Role Authentication:** Role-based registration and login with real-time duplication checks across Email and Username for all 4 roles.
- **Dynamic Homepage:** Custom banners, statistics, and role-adaptive hero buttons (e.g. Riders see their live delivery count, Restaurants see order tickets, Customers see the menu).
- **Multi-Vendor Cart & Checkout:** Customers can order from multiple restaurants in a single checkout. The system automatically splits the master order into separate restaurant sub-orders.
- **Automated Payment Gateway Mockup:** Full payment simulation supporting UPI (GPay, PhonePe, Paytm), Credit/Debit Cards, and NetBanking with transaction ledger logging.

- **Smart Automated Rider Dispatch:** Optimizes pickup batches and minimizes delivery SLA using distance, capacity, and restaurant affiliation scores.
- **Live Order Tracking with Real-Time Timeline:** An 8-stage interactive visual timeline with estimated ready times, kitchen stops, and real-time status history.
- **Secure OTP Verification:** Delivery is only completed when the rider inputs the customer's secret 6-digit confirmation OTP.
- **Performance Management System (PMS):** Tracks kitchen staff dish preparation count, PMS ratings, and dish submission approval workflows.

🛠 Technology Stack

Layer	Technology
Framework	ASP.NET MVC 5, .NET Framework 4.8.1
Language	C# 8.0+
Database	Microsoft SQL Server (Transact-SQL)
Data Access	ADO.NET (`SqlCommand` , `SqlDataReader` , Parameterized Stored Procedures)
Front-End UI	Razor Views (`.cshtml`), Bootstrap 5.2.3, Bootstrap Icons
Client Scripts	jQuery 3.7.0, jQuery Unobtrusive Validation, AJAX
Security	SHA-256 / Custom Password Hashing, CSRF Custom Tokens, Role Authorization

---

📁 Architecture & Project Structure

TEXT

```
FoodArrival/
│
├── App_Start/
│   ├── BundleConfig.cs          # Asset bundling (CSS/JS)
│   ├── FilterConfig.cs          # Global filters & exception handling
│   └── RouteConfig.cs           # MVC default route configurations
│
├── Controllers/
│   ├── HomeController.cs        # Landing page, dynamic hero, about, contact
│   ├── AccountController.cs     # 4-Role Registration & Login with validation
│   ├── FoodController.cs        # Menu catalog, category tabs, food search
│   ├── OrderController.cs       # Cart management, checkout, order history
│   ├── PaymentController.cs     # Dummy payment gateway, transaction recorder
│   ├── OrderTrackingController.cs # Live 8-stage visual order tracking
│   ├── RestaurantController.cs  # Restaurant dashboard, orders queue, menu CRUD
│   ├── StaffController.cs       # Kitchen staff portal, PMS, dish submissions
│   ├── RiderController.cs       # Delivery partner portal, order dispatch, OTP verify
│   └── AdminController.cs       # Executive dashboard, approvals, financials, bulletins
│
├── DAL/ (Data Access Layer)
│   ├── DbHelper.cs              # SQL connection provider and query executor
│   ├── CustomerRepository.cs    # Customer database access
│   ├── AdminRepository.cs       # Admin dashboard counts and reporting
│   ├── RestaurantRepository.cs  # Restaurant profile and kitchen management
│   ├── RiderRepository.cs       # Rider dispatch, status, and OTP verification
│   ├── StaffRepository.cs       # Kitchen staff onboarding, PMS rating, menu items
│   ├── FoodItemRepository.cs    # Food items catalog and category filters
│   ├── OrderRepository.cs       # Master orders, sub-orders, and SP wrappers
│   └── NotificationRepository.cs# In-app role-based alerts and unread counters
│
├── Services/
│   ├── IOrderTrackingService.cs   # Order tracking contract
│   ├── OrderTrackingService.cs    # 8-step timeline and ETA calculator
│   ├── IRiderAllocationService.cs # Dispatch engine contract
│   └── RiderAllocationService.cs  # Two-stage rider allocation & batching algorithm
│
├── Models/                      # Typed Domain Models and ViewModels
│   ├── Customer.cs, Admin.cs, Restaurant.cs, Rider.cs, RestaurantStaff.cs
│   ├── FoodItem.cs, Order.cs, OrderDetail.cs, OrderSubOrder.cs
│   ├── Transaction.cs, Bulletin.cs, Notification.cs, RiderAffiliation.cs
│   └── LoginViewModel.cs, OrderTrackingViewModel.cs, AdminDashboardViewModel.cs
│
├── Views/                       # Clean Razor UI pages
│   ├── Account/                 # Login & Registration with tabbed multi-role UI
│   ├── Home/                    # Landing page, About, Contact
│   ├── Food/                    # Food catalog with live filters
│   ├── Order/                   # PlaceOrder cart, order details
│   ├── OrderTracking/           # Live interactive order tracking screen
│   ├── Restaurant/              # Restaurant owner operations dashboard
```

```
│   ├── Staff/                  # Kitchen staff portal & PMS rating
│   ├── Rider/                  # Delivery partner dashboard & route view
│   ├── Admin/                  # System control center, approvals, finance, bulletins
│   └── Shared/                 # _Layout.cshtml with role-adaptive navbar
│
├── Content/                    # Stylesheets (Site.css, Bootstrap) and images
└── Web.config                  # Connection strings, app settings, runtime bindings
```

👥 User Roles & Complete Journeys

1. Customer Experience

- **Browse Menu:** Search food items by category (Pizza, Burger, Indian, Chinese, Snacks, Drinks).
- **Add to Cart:** Add items from single or multiple restaurants; adjust quantity via AJAX.
- **Checkout:** Enter delivery address and phone number.
- **Payment Gateway:** Choose UPI, Card, or NetBanking to simulate instant authorization.
- **Live Tracking:** Watch order move through 8 distinct steps with kitchen prep and rider ETA.
- **Order Receipt & OTP:** Receive a secret 6-digit Delivery OTP to share with the rider at the doorstep.

2. Restaurant Owner Portal

- **Onboarding:** Register with restaurant details, owner name, cuisine, address, and phone (verified by Admin).
- **Order Management:** View incoming orders, accept, move to `Preparing`, and mark `ReadyForPickup`.
- **Menu Management:** Add dishes, set prices, update descriptions, and toggle item availability.
- **Staff Management:** View and approve kitchen staff who registered under this restaurant.
- **Rider Fleet:** View delivery partners affiliated with this kitchen.
- **Financial Reports:** View earnings, platform deductions, and transaction history.

3. Kitchen Staff / Chef Portal

- **Registration:** Register by selecting a restaurant; system generates a unique Staff Code (e.g. `STF-REST1-001`).
- **PMS Rating:** Track personal Performance Management System rating (e.g. 4.9★) and total orders prepared.
- **Dish Submission:** Chefs can submit new dish ideas with price and image; these remain pending until approved by the Restaurant Owner or Admin.
- **Kitchen Queue:** View tickets and preparation countdown timers.

4. Delivery Partner / Rider Portal

- **Onboarding:** Register with vehicle type (Motorbike, Bicycle, Scooter, EV) and optional Driving License details.
- **Multi-Restaurant Affiliations:** Apply to multiple restaurants' delivery fleets.
- **Active Duty Toggle:** Switch status between `Available`, `On Delivery`, and `Offline`.

- **Order Pickup & Navigation:** View available delivery requests, restaurant stops, and customer delivery address.
- **OTP Handover:** Input the customer's 6-digit confirmation OTP to verify delivery completion.

5. System Administrator Control Center

- **Executive Dashboard:** Live metrics for pending orders, active customers, restaurants, riders, staff, and total platform revenue.
- **Approval Hub:** Approve newly registered restaurants, kitchen staff, rider affiliations, and staff-submitted menu items.
- **Financials & Ledger:** Monitor total transaction volume, platform fee revenue, and restaurant disbursements.
- **Public Bulletins:** Broadcast urgent announcements, weather warnings, or promotions to targeted audiences (`All` , `Customers` , `Restaurants` , `Riders`).

🧠 Intelligent Backend Algorithms & Engines

Two-Stage Smart Rider Allocation Service

Located at `Services/RiderAllocationService.cs` :

- **Stage 1 (Feasibility & Batching):** Calculates pickup distance and ETA gap. If multiple restaurant stops can be completed by one rider within SLA, it creates a single multi-stop batch.
- **Stage 2 (Candidate Scoring):** Scores available riders using a weighted composite formula:

$$\text{Score} = 0.25(\text{Distance}) + 0.25(\text{ETA}) + 0.15(\text{Workload}) + 0.15(\text{Route}) + 0.10(\text{Affiliation}) + 0.10(\text{Vehicle})$$

Live 8-Stage Order Tracking & Route Map

Located at `Services/OrderTrackingService.cs` and `Views/OrderTracking/Track.cshtml` :

- `Order Placed`
- `Payment Confirmed`
- `Preparing Food`
- `Ready for Pickup`
- `Rider Assigned`
- `Picked Up from Kitchen`
- `Out for Delivery`
- `Delivered to Doorstep`

6-Digit Secure Delivery OTP Handover

- On checkout, a randomized 6-digit OTP is generated and saved in `Orders.DeliveryOtp` .
- The customer sees this code on their live tracking screen.
- The rider cannot mark an order as `Delivered` without inputting this matching code in `RiderController.CompleteDelivery` .

5-Minute Inactive Order Auto-Rejection

- Stored procedure `dbo.sp_Order_AutoReject` automatically rejects any pending orders if unacknowledged by the restaurant for 5 minutes (`DATEDIFF(SECOND, OrderedAt, GETDATE()) >= 300`).

Multi-Vendor Cart Splitting

- An order with items from multiple restaurants is stored as one master order in `dbo.Orders` , with child records generated in `dbo.OrderSubOrders` for each restaurant.

🗄 Database Schema & Scripts

All SQL scripts have been separated table-wise and stored in `D:\avi\sql\FoodReady\` :

Script File	Purpose
`00_README_And_Master_Execution_Order.sql`	Master execution instructions and database creation
`01_Customers.sql`	Customer accounts, authentication SPs, and queries
`02_Admins.sql`	Administrator accounts, dashboard metric queries, and credentials
`03_Restaurants.sql`	Restaurant profiles, cuisine catalog, and approval queries
`04_Riders.sql`	Delivery partner accounts, vehicle details, and status queries
`05_RestaurantStaff.sql`	Kitchen staff, Staff Code generation, and PMS rating queries
`06_FoodItems.sql`	Menu catalog, food category filters, and dish approval queries
`07_Orders.sql`	Master orders, TVP `dbo.OrderItemType` , and lifecycle SPs
`08_OrderDetails.sql`	Line items with persisted `SubTotal` computed column
`09_Notifications.sql`	Role-targeted notification alerts and unread badge counters
`10_RiderRestaurantAffiliations.sql`	Rider-to-restaurant fleet mapping and license verification
`11_Transactions.sql`	Payment gateway ledger, platform fee, and restaurant share
`12_Bulletins.sql`	Admin broadcast announcements by audience (`All` , `Riders` , etc.)
`13_OrderSubOrders.sql`	Multi-vendor cart split sub-orders per restaurant
`14_OrderStatusHistory.sql`	Immutable audit trail for order timeline events

(Alternatively, the single consolidated script is available at `FoodDeliveryDB_All_SQL.sql` in the project root).

🔑 Default Seed Credentials for Testing

Role	Username / Login ID	Password	Portal URL
System Admin	`avi@123` *(or* `admin@example.com` *)*	`28072003` *(or* `admin123` *)*	`/Admin/Dashboard`
Restaurant Owner	`restaurant@example.com`	`rest123`	`/Restaurant/Dashboard`
Kitchen Staff	`chef.vikram@example.com` *(Code:* `STF-REST1-001` *)*	`chef123`	`/Staff/Dashboard`
Delivery Rider	`rider@example.com` *(speedy_amit)*	`rider123`	`/Rider/Dashboard`
Customer	`john@example.com`	`12345`	`/Food/Index`

🚀 How to Run & Setup Locally

Prerequisites

- **Visual Studio 2022 / 2019** with the *.NET desktop development* and *ASP.NET and web development* workloads installed.
- **Microsoft SQL Server** (LocalDB, Express, or Developer edition).
- **SQL Server Management Studio (SSMS)** or Azure Data Studio.

Step-by-Step Setup

- **Initialize Database:**
- Open SSMS and connect to your SQL Server instance (`(localdb)\MSSQLLocalDB` or `.` / `localhost`).
- Run the master script `FoodDeliveryDB_All_SQL.sql` (or run scripts in order from `D:\avi\sql\FoodReady\01_.sql` *to* `14_` `.sql`).
- This creates `FoodArrivalDB` with all tables, constraints, stored procedures, and sample data.

- **Verify Connection String:**
- Open `Web.config` in the root of `FoodArrival` .
- Ensure the connection string matches your local SQL Server instance:

XML

```
    <connectionStrings>
      <add name="FoodArrivalDB"
           connectionString="Data Source=(localdb)\MSSQLLocalDB;Initial Catalog=FoodArrivalDB;In
           providerName="System.Data.SqlClient" />
    </connectionStrings>
```

- **Open & Build Project:**
- Open `FoodArrival.slnx` (or `FoodArrival.csproj`) in Visual Studio.
- Restore NuGet packages: `Tools -> NuGet Package Manager -> Restore NuGet Packages` .

- Build Solution (`Ctrl + Shift + B`).
- **Launch Application:**
- Press `F5` or click `IIS Express (Google Chrome / Edge)`.
- Navigate to `http://localhost:port/` to start exploring!